

SQL Injection

4/25/26

11:09 PM

What it is

- **SQL Injection (SQLi)** = user input becomes part of SQL query → attacker controls query.
- Root cause: **string concatenation with untrusted input**

Vulnerable pattern

```
SELECT * FROM articles WHERE id = <user_input>
```

Attack example

```
99'; DROP TABLE articles; --
```

Key SQL Characters (used in attacks)

- ' → break out of string
- -- / # / /* */ → comments (ignore rest of query)
- %, _ → wildcards (LIKE)
- () → grouping
- \ → escape

Prevention (what to expect)

-  Escaping → weak
-  **Parameterized queries / prepared statements** (best)
-  ORM (if not using raw SQL)

Types of SQL Injection

1. In-band (most common)

- Immediate feedback (errors/data)
- Used in most demos

2. Blind (inferential)

- No direct output
- Use:
 - response time
 - behavior differences

3. Out-of-band (rare)

- DB interacts externally
 - write files
 - HTTP requests

Detection Workflow (VERY IMPORTANT)

1. **Find inputs**
 - GET, POST, headers
2. **Send payload**
 - ', OR 1=1, UNION
3. **Analyze**
 - errors?
 - different output?
4. **Confirm exploit**
 - login bypass
 - data access
 - side effects

Demo Techniques (What You Need to Know)

1. Login Bypass (Classic)

Payloads:

```
' --
```

→ comments out password check

```
' OR 1=1 --
```

→ always true → logs in as first user (often admin)

```
' OR "" --
```

Key Idea:

- Break query logic → force TRUE condition

2. UNION Attack (Extract / Fake Data)

Goal:

- Combine attacker-controlled data with real query

Steps:

1. Find column count:

```
' UNION SELECT 1,2,3 --
```

→ adjust until no error

2. Inject data:

```
' UNION SELECT 'user','pass',1 --
```

Result:

- App trusts DB → logs in as fake user

3. Retrieve Hidden Data

Bypass filters:

```
' OR 1=1 --
```

Remove conditions:

```
' /*
```

→ comments out restrictions like:

```
AND isAvailable = 1
```

Result:

- Hidden/filtered data exposed

4. Extract Passwords (Advanced)

Technique: Boolean-based extraction

```
' AND password > '7' --
```

Idea:

- Compare values → infer characters
- Repeat → reconstruct hash

Faster method:

```
UNION SELECT password AS name ...
```

→ display password hash directly in UI

5. Blind SQL Injection

No errors → use behavior

Time-based:

```
AND benchmark(5000000, sha1('test')) --
```

→ slow response = success

Logic-based:

```
7-4
```

→ evaluated as 3 → proves injection works

6. Out-of-Band SQLi

Example (MySQL):

```
SELECT 'php code' INTO OUTFILE 'c:/path/file.php'
```

Result:

- Writes file to server
- Can execute code via web server

NoSQL Injection (MongoDB)

Concept:

- Still query-based → still injectable

Example:

```
?id[$gt]=3
```

→ becomes:

```
{ id: { $gt: 3 } }
```

Result:

- Returns unintended data (e.g., hidden records)

ORM Injection (IMPORTANT for exams/interviews)

Even ORMs can be vulnerable if using raw queries:

Red flags:

- .raw()
- ExecuteRawQuery
- FromSqlRaw
- createQuery() with string concat
- createSQLQuery()

Rule:

If user input is concatenated → SQLi still possible

Key Attack Patterns (Memorize)

- ' OR 1=1 --
- ' --
- ' UNION SELECT ... --
- ' AND condition --
- /* comment */

What Actually Matters (TL;DR)

- SQLi = **user input** → **SQL query**
- Look for:
 - login forms
 - search fields
 - IDs in URLs
- Use:
 - ' → test
 - OR 1=1 → bypass
 - UNION → extract data
- Blind:
 - time delays
 - logic tricks
- Always confirm:
 - login bypass
 - extra data
 - side effects